

Evidencia Implementación de Análisis Léxico (Autómata y Expresión Regular)

Carlos Delgado Contreras

A01712819

18 Marzo de 2026

Implementación de métodos computacionales (Gpo 603)

Lenguaje Seleccionado

Para este Análisis Léxico seleccionamos el siguiente conjunto de palabras pertenecientes al lenguaje Chakobsa del Universo literario de Dune de Frank Herbert. El Chakobsa es un lenguaje antiguo que funge como base de otros lenguajes de diferentes civilizaciones alrededor del imperio. Desde los Nómadas Fremen, la familia real Atreides hasta el enigmático culto Bene Gesserit combinan el dialecto Chakobsa con otros lenguajes. Originalmente, era un "lenguaje de caza" utilizado por los Bhotani, una antigua hermandad de asesinos durante las Guerras de Asesinos. Desde el título Fremen de “Lisan al-Gaib” hasta el nombre de los asentamientos “Sietch” son provenientes del Chakobsa.

El conjunto de palabras a analizar e identificar es el siguiente:

- A. chaumas - poison in solid food (imported from Rima)
- B. chaumurky, musky, murky - poison in drink (imported from Ishkal)
- C. cherem - brotherhood of hatred (imported from Ishkal)
- D. chouhada - purposeful fighters (imported from Ishkal)
- E. cielago - (from older Harmonthepic compound ciel "water" and lako "fowl"; or more probably from "murciélago", bat in Spanish)

Expresión Regular (REGEX)

Se planteó una expresión regular utilizando metacaracteres que exprese el patrón de búsqueda exacto para poder identificar este & solo este conjunto de palabras de manera estricta.

Regex:

```
^[Cc](h(aum(as|urky)$|er(em)$|ouh(ada)$)|i(elago)$)
```

```
: / ^[Cc](h(aum(as|urky)$|er(em)$|ouh(ada)$)|i(elago)$)
```

Test String

```
chaumas ↵  
Chaumurky ↵  
cherem ↵  
Chouhada ↵  
cielago ↵  
↵  
Cheumas ↵  
Chaumerky ↵  
cheremm ↵  
Couhada ↵  
cielaga ↵  
ChAumas ↵
```

Regex a NFA

Lo primero que se debe de hacer es simplificar en Regex a tal punto que solo tengamos las unidades básicas que las reglas de thompson puedan traducir:

```
^(C|c)(h(aum(as|urky)$|er(em)$|ouh(ada)$)|i(elago)$)
```

```
:/ ^((C|c)(h(aum(as|urky)$|er(em)$|ouh(ada)$)|i(elago)$)) / gm

Test String

Chaumas
Chaumurky
cherem
chouhada
Cielago
Cheumas
chaumass
Chiumurky
Ccherem
Ccielago
CHaumurky
```

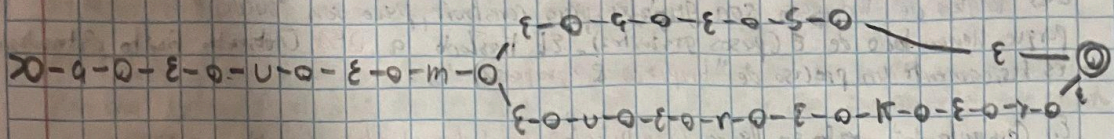
Traducción a NFA & Reglas de Thompson

Para la traducción de Regex a NFA y DFA se utilizaron las Reglas de McNaughton-Yamda-Thompson así como lo estableció Alfred V. et al. En su libro Compilers capítulo 3.7.4. Esto con el objetivo de transformar de manera correcta y precisa los metacaracteres del Regex en Estados y Transiciones válidas de un NFA.

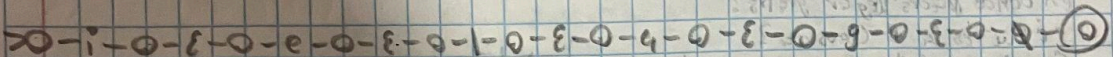
Es importante aclarar que las reglas de Thompson por su naturaleza No Determinista (Transiciones Epsilon) producen exclusivamente un autómata tipo NFA. Las reglas de Thomson utilizadas en esta conversión fueron:

1. Unidad Inicial (a): Estado inicial se conecta con estado final mediante una Transición de Carácter del Alfabeto.
2. Concatenación (ab): Los autómatas se conectan mediante una Transición Epsilon donde desde el estado final del primero hasta el estado inicial del segundo con convirtiendo a ambos en estados estándar

- Gum (as Turkey) \$

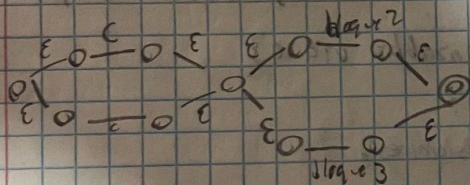


3 bloque 3: relagos



- bifurcación desde bloque 1°.

(c/c) (bloque 2 | bloque 3)

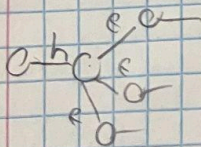
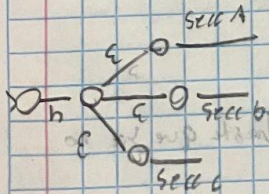


* Triple rate em h (bloque 2)

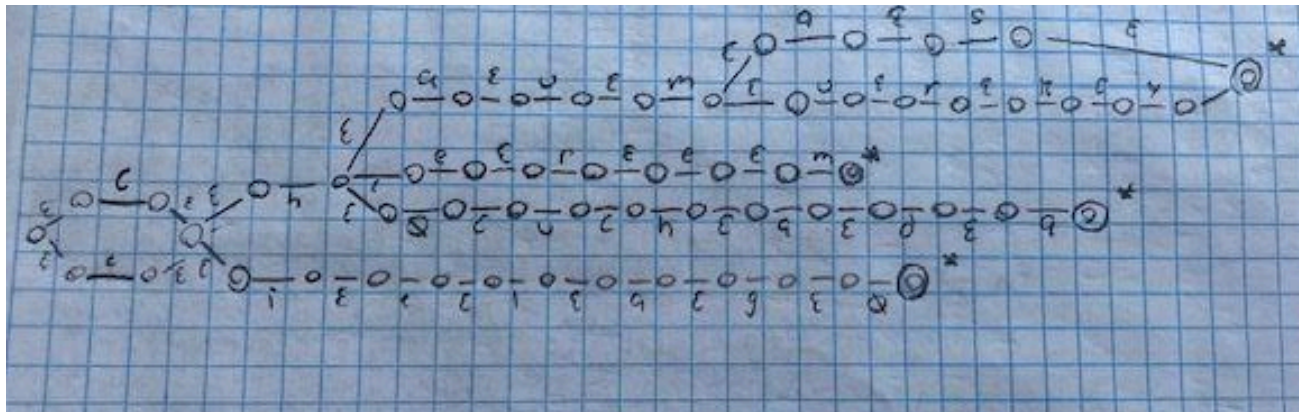
- section A (geom (45/urky))

- Secrete B (erectile)

oseccen c (ouhede t)



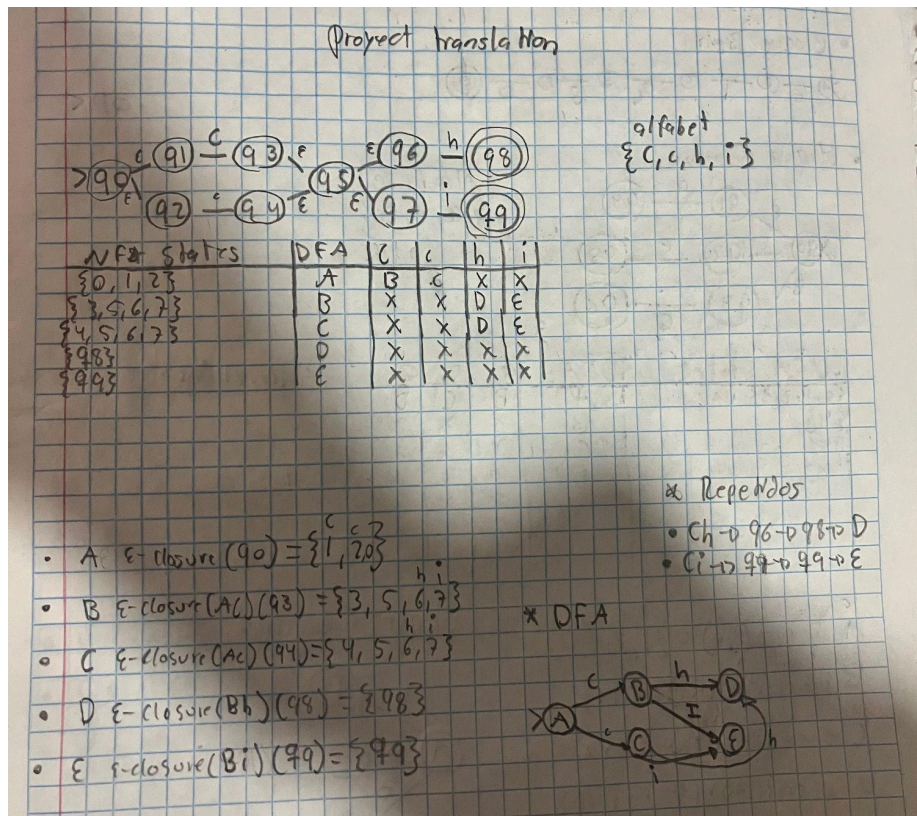
Una vez obtenido todos los bloques se unifican obteniendo el siguiente NFA completo:



Traducción Mediante Construcción de Subconjuntos

Según Alfred V. et al. En el capítulo 3.7 del libro Compilers (2006) la naturaleza no determinística de los NFA hace que su simulación en código sea más compleja de realizar. Al tener transiciones válidas al mismo tiempo en un mismo estado, como por ejemplo los epsilon, se debe de resolver el no determinismo lo cual agrega una capa de dificultad a la codificación. Una de las soluciones propuestas por Alfred en el capítulo 3.7 de su libro, la cual se utiliza en este proyecto, es la Construcción de Subconjuntos. La premisa detrás de este método es proponer estados “Globales” que unifiquen estados del NFA. Reduciendo de esta forma el número de estados y limpiando el autómata de las transiciones epsilon. Para lograr esto se identifican las **Clausuras Epsilon** (e-closure) las cuales son el **conjunto de estados** a los que se puede llegar desde un estado solo con movimientos epsilon. Por último proponemos una **Tabla de transiciones** en la que enlistamos los nuevos estados DFA con las transiciones pertinentes a cada elemento de nuestra gramática.

A continuación se muestra un ejemplo de traducción de un bloque NFA a DFA:



Una vez traducido todos los bloques del NFA a DFA mediante subconjuntos obtenemos el siguiente DFA completo:

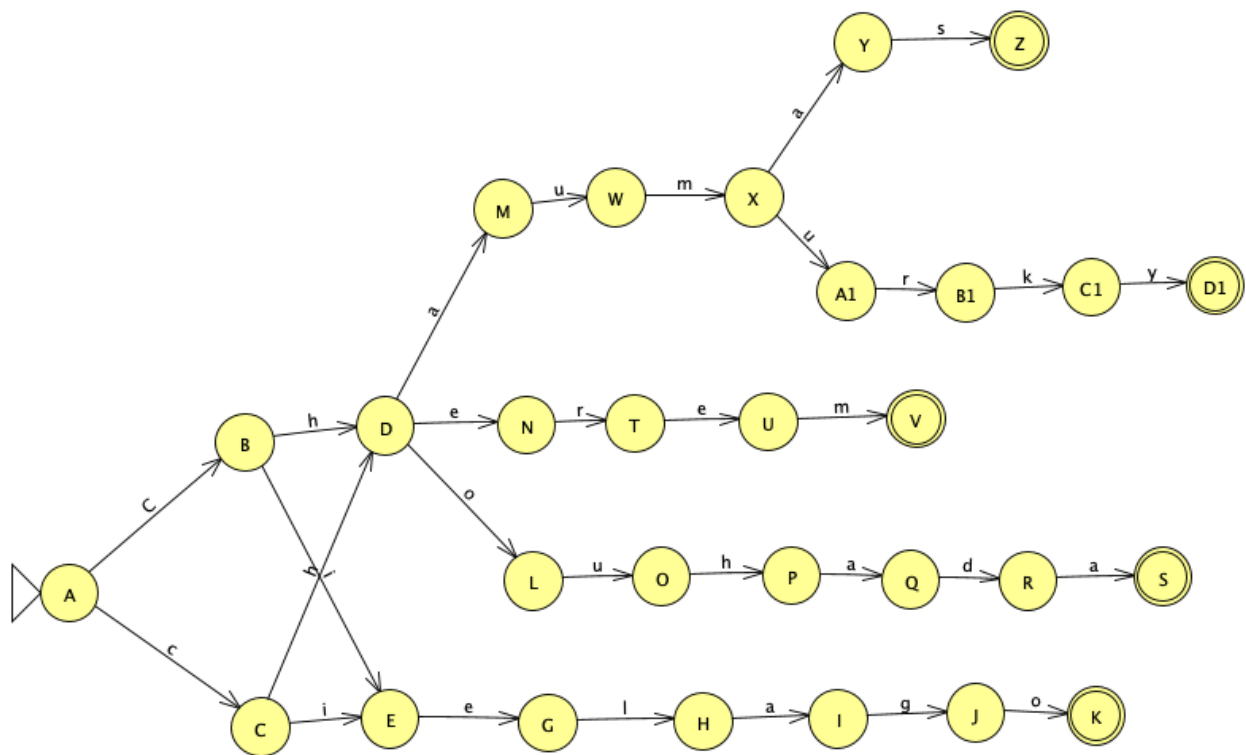


Tabla de Estados y Transiciones.

Estado Origen	Carácter	Estado Destino	Palabra / Rama correspondiente
A (qa)	C	B (qb)	Inicio (Mayúscula)
A (qa)	c	C (qc)	Inicio (Minúscula)
B (qb)	h	D (qd)	Rama 'Ch...'
B (qb)	i	E (qe)	Rama 'Ci...'
C (qc)	h	D (qd)	Rama 'ch...'
C (qc)	i	E (qe)	Rama 'ci...'
D (qd)	a	M (qm)	Prefijo 'Cha...'
D (qd)	e	N (qn)	Prefijo 'Che...'
D (qd)	o	L (ql)	Prefijo 'Cho...'

E (qe)	e	G (qg)	Prefijo 'Cie...'
G (qg)	i	H (qh)	Ciel...
H (qh)	a	I (qi)	Ciela...
I (qi)	g	J (qj)	Cielag...
J (qj)	o	Z (qz)	Cielago (Final)
L (ql)	u	O (qo)	Chou...
O (qo)	h	P (qp)	Chouh...
P (qp)	a	Q (q_q)	Chouha...
Q (q_q)	d	R (q_r)	Chouhad...
R (q_r)	a	Z (qz)	Chouhada (Final)

N (qn)	r	T (qt)	Cher...
T (qt)	e	U (qu)	Chere...
U (qu)	m	Z (qz)	Cherem (Final)
M (qm)	u	W (q_w)	Chau...
W (q_w)	m	X (q_x)	Chaum...
X (q_x)	a	Y (qy)	Chauma...
X (q_x)	u	A1 (qa1)	Chaumu...
Y (qy)	s	Z (qz)	Chaumas (Final)
A1 (qa1)	r	B1 (qb1)	Chaumur...
B1 (qb1)	k	C1 (qc1)	Chaumurk...

C1 (qc1)	y	Z (qz)	Chaumurky (Final)
-----------------	----------	---------------	--------------------------

Pruebas de DFA en JFLAP

Se utilizó JFLAP para tanto la unificación del DFA como la realización de pruebas previas a la codificación.

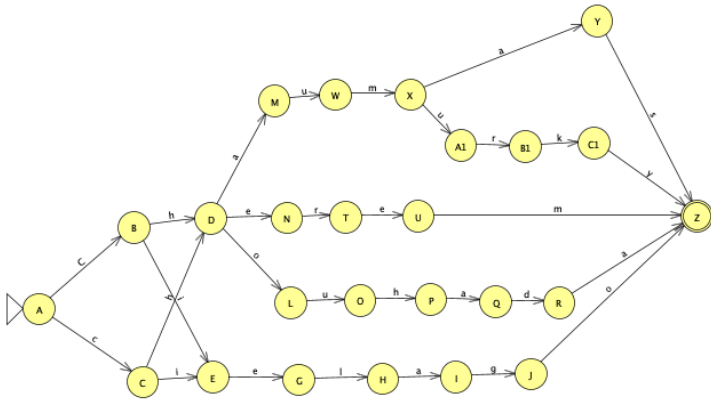


Table Text Size

Input	Result
chaumas	Accept
chaumurky	Accept
cherem	Accept
chouhada	Accept
cielago	Accept
charam	Reject
chaumass	Reject
chherem	Reject
cielego	Reject
cheuhede	Reject
Chaumas	Accept
Chaumurky	Accept
Cherem	Accept
Chouhada	Accept
Cielago	Accept

Codificación de DFA a Prolog

Se utilizó Prolog para la codificación y navegación del DFA. Partimos de generar un listado con nuestro estado inicial, nuestras transiciones en formato (estadoOrigen, Caracter, estadoDestino) & nuestros estados finales.

```
%Starting State.
start_state(qa).

%Transition list
%transition(estadoOrigen, 'caracter', estadoDestino).
transition(qa, 'C', qb).
transition(qa, 'c', qc).
transition(qb, 'h', qd).
transition(qb, 'i', qe).
transition(qc, 'h', qd).
transition(qc, 'i', qe).
transition(qd, 'a', qm).
transition(qd, 'e', qn).
transition(qd, 'o', ql).
transition(qe, 'e', qg).
transition(qg, 'l', qh).
transition(qh, 'a', qi).
transition(qi, 'g', qj).
transition(qj, 'o', qz).
transition(ql, 'u', qo).
transition(qo, 'h', qp).
transition(qp, 'a', q_q).
transition(q_q, 'd', q_r).
transition(q_r, 'a', qz).
transition(qn, 'r', qt).
transition(qt, 'e', qu).
transition(qu, 'm', qz).
transition(qm, 'u', q_w).
transition(q_w, 'm', q_x).
transition(q_x, 'a', qy).
transition(q_x, 'u', qa1).
transition(qy, 's', qz).
transition(qa1, 'r', qb1).
transition(qb1, 'k', qc1).
transition(qc1, 'y', qz).

%Finish State.
end_state(qz).
```

Una vez establecidas las transiciones, codificamos nuestro motor de navegación conformado por un Parser de Inputs, la navegación de transiciones y un verificador de estado final.

Para poder recibir un Input del jugador y poder manipularlo como una lista utilizamos la función nativa de Prolog:

```
atom_chars(P,L)
```

Dicha función recibe una variable y la convierte en una lista con Head & Tail.

La navegación utiliza recursión por cola primero llamando a la transición y después la llamada recursiva. Como caso base establecemos que la navegación ya no cuenta con más letras por procesar en transiciones. La verificación de estado final consiste en tratar de

unificar el estado final con el estado actual. Si es posible nos encontramos en un estado final y la palabra es correcta. De lo contrario no llegaríamos a una palabra del diccionario Chakobsa.

```
%Motor de navegacion.

chakobsa(P):-
    atom_chars(P,L),
    start_state(S),

    %Outputs personalizados.
    (navegation(S,L) ->
        write("¡Bi-la Kaifa! palabra sagrada del Chakobsa")
        ;
        write('lengua de espías o simples extranjeros ¡No es Chakobsa!')
    ).

%Recursive call.
navegation(State,[H|T]):-
    transition(State,H,Newstate),
    navegation(Newstate,T).

%Base Case: No more letters in list.
navegation(State,[]):-
    end_state(State).
```

Pruebas Prolog

Palabra	Aceptado/Rechazado
chaumas	Aceptado ▾
chaumurky	Aceptado ▾
cherem	Aceptado ▾
chouhada	Aceptado ▾
cielago	Aceptado ▾
charam	Aceptado ▾
chaumass	Aceptado ▾
chherem	Aceptado ▾
cielego	Aceptado ▾
cheuhede	Aceptado ▾
	Aceptado ▾
CCCC	Aceptado ▾
Chaumas	Aceptado ▾
Chaumurky	Aceptado ▾
Cherem	Aceptado ▾
Chouhada	Aceptado ▾
Cielago	Aceptado ▾

Pruebas Automáticas

para poder ejecutar la prueba automatica se debe de ejecutar `run_test` en Prolog el cual ya cuenta con un listado de `test_dato(palabra, resultado)`. En caso que se quiera añadir más pruebas a la ejecución debe de añadir un predicado nuevo a la lista.

```
112 ?- run_test.  
  
=== INICIANDO PRUEBAS DEL SISTEMA CHAKOBSA ===  
  
Palabra: chaumas | Esperado: true | Obtuvimos: true | [PASÓ]  
Palabra: chaumurky | Esperado: true | Obtuvimos: true | [PASÓ]  
Palabra: cherem | Esperado: true | Obtuvimos: true | [PASÓ]  
Palabra: chouhada | Esperado: true | Obtuvimos: true | [PASÓ]  
Palabra: cielago | Esperado: true | Obtuvimos: true | [PASÓ]  
Palabra: charam | Esperado: false | Obtuvimos: false | [PASÓ]  
Palabra: chaumass | Esperado: false | Obtuvimos: false | [PASÓ]  
Palabra: chherem | Esperado: false | Obtuvimos: false | [PASÓ]  
Palabra: cielego | Esperado: false | Obtuvimos: false | [PASÓ]  
Palabra: cheuhede | Esperado: false | Obtuvimos: false | [PASÓ]  
Palabra: | Esperado: false | Obtuvimos: false | [PASÓ]  
Palabra: CCCC | Esperado: false | Obtuvimos: false | [PASÓ]  
Palabra: Chaumas | Esperado: true | Obtuvimos: true | [PASÓ]  
Palabra: Chaumurky | Esperado: true | Obtuvimos: true | [PASÓ]  
Palabra: Cherem | Esperado: true | Obtuvimos: true | [PASÓ]  
Palabra: Chouhada | Esperado: true | Obtuvimos: true | [PASÓ]  
Palabra: Cielago | Esperado: true | Obtuvimos: true | [PASÓ]  
  
=== FIN DE LAS PRUEBAS ===  
true.
```

Dichas pruebas verifican que el algoritmo en Prolog acepta las palabras de Chakobsa tanto con inicial en mayúscula como con minúscula. Rechaza palabras no pertenecientes al conjunto del Chakobsa al igual que nulos. Se confirma el correcto funcionamiento del programa en Prolog y su validez con el DFA hecho en JFLAP

Análisis de Complejidad y Soluciones Alternas

Alfred V. et Al explican en la sección 3.7.5 la diferencia de eficiencia de procesamiento de string entre la simulación de un DFA y un NFA. Se establece que un DFA al tener determinismo en cada estado (1 estado a la vez simultáneamente) el algoritmo procesa una cadena de string x con una complejidad asintótica de $O(|x|)$ dándole un crecimiento **lineal**. Es decir que si un DFA debe de procesar una cadena de string de 10 mil caracteres el algoritmo tomará 10 mil pasos para poder procesarlo pues cada estado solo tiene 1 estado posible a transicionar. Por otro lado los NFA por su naturaleza no determinística cada estado tiene múltiples transiciones que son posibles **al mismo tiempo**. El algoritmo debe de recorrer cada ruta posible para poder finalizar el recorrido. La complejidad asintótica escala a un producto entre los caracteres del string (x) por el **tamaño** de la **Tabla de transiciones del NFA**: $O(|x| * NFA\ size)$

Esto cambia el comportamiento lineal a un crecimiento **Polinomial**.

A primera vista pareciese que simular un NFA no es viable en ninguna circunstancia teniendo el comportamiento lineal de los DFA. No obstante hay un matiz a tener en cuenta. Un DFA siempre surge de un NFA mediante Construcción de Subconjuntos y este proceso en el pero de los casos puede **exponencialmente** aumentar la cantidad de estados resultantes de la traducción haciendo de la implementación del DFA prácticamente inviable. Esto sucede porque el DFA resultante de la construcción de subconjuntos debe de representar **todas** las combinaciones de los estados del NFA (2^n). El número de estados de un DFA puede llegar a los millones si nos encontramos ante un NFA demasiado complejo y extremadamente No Determinista. Son en estas situaciones donde es más viable simular el NFA sobre un DFA. En este proyecto el NFA resultante no contaba con demasiada ambigüedad de estados simultáneos. El no determinismo fue dado por las reglas de Thompson más que por la naturaleza del contexto. Por lo que la construcción de Subconjuntos dio en un caso positivo y deseable reduciendo la cantidad de estados

Como resultado, el DFA presentado cuenta con una complejidad lineal de $O(|x|)$.

Referencias y Bibliografías

Alfred V. Aho, Monica S. Lam, Ravi Sethi y Jeffrey D. Ullman (2006). *Compilers: Principles, Techniques, & Tools* (2^a ed.). Editorial Addison-Weasley Pearson.

